

UniCaronas — Sistema de Caronas Universitárias

Resumo da Reunião de Planejamento — Sprint 7

Engenharia de Software — 3ESCN

Período: 08/05/2026 a 14/05/2026 (1 semana)

Função	Nome
Product Owner — define e esclarece os requisitos	Rafael Machado
Scrum Master — facilita a cerimônia	Ariane da Silva Archanjo
Dev Team — estima, planeja e se compromete	Pedro Kafka, Matheus Sizanoski, Rafael Machado, Ariane da Silva Archanjo

1. Como a cerimônia aconteceu

A reunião de Sprint Planning foi realizada em 07/05/2026 de forma presencial. A Scrum Master conduziu o planejamento e o Product Owner apresentou os itens prioritários para esta sprint. Com base na decisão técnica tomada na Sprint 6, o foco principal desta sprint é a validação do e-mail universitário via código/token enviado por e-mail — substituindo a dependência da API ID Sheet por uma solução gratuita e robusta baseada em lista local de domínios e Nodemailer. Paralelamente, a equipe dedicará esforço à refatoração do código, ao Dashboard de Impacto (Eco-Stats) e à Exportação de Comprovantes em PDF.

Campo	Detalhe
Data	07/05/2026 (quinta-feira)
Formato	Presencial
Duração	Aproximadamente 1h30
Quem conduziu	Ariane da Silva Archanjo (Scrum Master)
Quem definiu as tarefas	Rafael Machado (Product Owner)
Início da Sprint	08/05/2026
Fim da Sprint	14/05/2026 (1 semana)
Caráter	Sprint de qualidade técnica — validação de e-mail institucional por token, refatoração de código, Eco-Stats e exportação de comprovantes — duração de 1 semana

2. O que será entregue ao final da Sprint

Ao final desta Sprint, o UniCaronas terá validação automática de vínculo universitário via código de confirmação enviado ao e-mail institucional do usuário — sem upload manual, sem revisão humana e sem dependência de APIs pagas. O backend verificará o domínio do e-mail contra uma lista local de domínios reconhecidos (JSON ou tabela no banco) e, se válido, enviará um código/link de ativação via Nodemailer (SMTP gratuito). O código do backend estará refatorado com controllers organizados e testes cobrindo os fluxos críticos. O perfil exibirá o Dashboard de Impacto com CO2 evitado e economia financeira. Caronas concluídas poderão ser exportadas como comprovante em PDF.

A entrega está completa quando:

- Domínio do e-mail é validado contra lista local; domínios não reconhecidos retornam erro 400
- Código/token de verificação é enviado ao e-mail institucional via Nodemailer
- Usuário confirma o código e a conta é ativada com email_verificado = true e instituicao_nome preenchido
- Usuários com e-mail pessoal ou domínio não reconhecido recebem mensagem clara orientando uso de e-mail institucional
- Painel admin exibe status de validação de cada usuário sem necessidade de revisão manual
- Controllers do backend organizados em módulos com responsabilidade única
- Respostas da API padronizadas — todos os erros seguem o formato { success, error, code }
- Testes unitários cobrindo os controllers de usuários, caronas, pagamentos e avaliações
- Dashboard de Impacto exibindo CO2 evitado (kg) e economia financeira acumulada no perfil
- Botão "Exportar comprovante" disponível em caronas concluídas gerando PDF formatado

3. Quais histórias de usuário serão trabalhadas

O Product Owner apresentou quatro histórias para esta sprint, combinando entrega técnica de qualidade com funcionalidades do roadmap.

ID	Funcionalidade	Pontos	Prioridade	Sprint
US22	Validação de e-mail universitário por código/token	5 pontos	Alta	7
US23	Refatoração do código e cobertura de testes	5 pontos	Alta	7
US24	Dashboard de Impacto — Eco-Stats no perfil	3 pontos	Média	7
US25	Exportação de comprovante em PDF	2 pontos	Média	7
—	Total	15 pontos	—	—

4. Por quem será feito

Membro	Responsabilidade principal
Rafael Machado	Product Owner: refinamento de requisitos, definição da lista de domínios institucionais, configuração do SMTP (Nodemailer) e contrato de resposta padronizado da API
Ariane da Silva Archanjo	Scrum Master: facilitação das cerimônias; desenvolvimento — refatoração dos controllers, padronização de respostas, testes unitários com Jest/Supertest e Dashboard de Impacto (backend + frontend)
Pedro Kafka	Desenvolvimento: fluxo completo de validação por e-mail — middleware de verificação de domínio, geração e envio de token, endpoint de confirmação, exibição de status no perfil e painel admin
Matheus Sizanoski	Desenvolvimento: exportação de comprovante em PDF, testes manuais do fluxo completo e atualização do README com variáveis de ambiente (SMTP_USER, SMTP_PASS, SMTP_HOST, etc.)

5. Como cada funcionalidade será feita

US22 — Validação de e-mail universitário por código/token (5 pontos)

Como administrador, quero que o sistema valide automaticamente se o e-mail do usuário pertence a uma instituição de ensino reconhecida, enviando um código de confirmação ao e-mail informado, sem necessidade de upload ou revisão manual.

Novo fluxo de verificação:

- No cadastro, o usuário informa seu e-mail institucional (ex: joao@aluno.unibrasil.com.br)
- O backend extrai o domínio e verifica contra a lista local de domínios permitidos (JSON ou tabela `dominios_permitidos` no PostgreSQL)
- Se o domínio não for reconhecido, retorna erro 400 com mensagem orientando uso de e-mail institucional válido
- Se o domínio for válido, o sistema gera um token aleatório (UUID ou 6 dígitos) com validade de 15 minutos e envia via Nodemailer (SMTP gratuito: Gmail, Outlook ou Brevo free tier)
- O usuário acessa o e-mail e insere o código ou clica no link de verificação
- Endpoint POST `/api/usuarios/verificar-email` valida o token; se correto e dentro do prazo, seta `email_verificado = true` e `instituicao_nome` no banco
- Tokens expirados retornam erro 400 com opção de reenvio via POST `/api/usuarios/reenviar-token`

Implementação técnica:

- Lista de domínios armazenada localmente em `domains.json` ou tabela PostgreSQL — sem dependência de API externa paga
- Middleware `validateDomainInstitucional` — executado no POST `/api/usuarios` — verifica domínio e bloqueia e-mails não reconhecidos
- Tabela `verification_tokens`: campos `usuario_id`, `token`, `expires_at`, `used` — gerenciada via ALTER TABLE
- Nodemailer configurado com SMTP gratuito — credenciais em `SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASS` no `.env`
- Painel admin exibe coluna "E-mail verificado" na listagem de usuários — sem ação manual do admin
- Variáveis de ambiente documentadas no README

Para considerar pronto:

Critério de Aceitação	Status
Domínio institucional reconhecido: sistema envia token por e-mail	■ Planejado
Domínio não reconhecido retorna erro 400 com mensagem clara	■ Planejado
Token confirmado dentro do prazo: <code>email_verificado = true</code> e <code>instituicao_nome</code> salvos no banco	■ Planejado
Token expirado retorna erro 400 com orientação de reenvio	■ Planejado
Painel admin exibe status de verificação de cada usuário sem revisão manual	■ Planejado

US23 — Refatoração do código e cobertura de testes (5 pontos)

Como equipe de desenvolvimento, queremos que o código esteja organizado, padronizado e coberto por testes para facilitar manutenção e novas features.

Como será feito:

- Controllers reorganizados: cada arquivo responsável por um único recurso — sem lógica de negócio misturada com queries SQL brutas
- Camada de serviço (services/) extraída dos controllers — caronasService.js, usuariosService.js — separando regras de negócio do HTTP
- Respostas da API padronizadas: todos os endpoints retornam { success: true/false, data: {}, error: string, code: string }
- Testes unitários com Jest: autenticação JWT, cálculo de taxa de pagamento, lógica de vagas, trigger de avaliação e validação de domínio institucional
- Testes de integração com Supertest: cadastro → verificação de e-mail por token → login → criar carona → solicitar → aceitar → pagar → avaliar
- Linting com ESLint configurado e sem warnings no repositório

Para considerar pronto:

Critério de Aceitação	Status
Camada de serviços extraída dos controllers — sem SQL direto nos arquivos de rota	■ Planejado
Todas as respostas da API seguem o formato padronizado { success, data, error, code }	■ Planejado
Cobertura de testes unitários $\geq 70\%$ nos arquivos de serviço	■ Planejado
Testes de integração do fluxo completo (incluindo verificação de token) passando sem erros	■ Planejado
ESLint configurado e sem warnings no repositório	■ Planejado

US24 — Dashboard de Impacto — Eco-Stats no perfil (3 pontos)

Como usuário, quero ver no meu perfil quanto de CO2 eu ajudei a evitar e quanto economizei financeiramente ao compartilhar caronas.

Como será feito:

- Endpoint GET /api/usuarios/:id/eco-stats — calcula: km totais em carona, CO2 evitado ($\text{km} \times 0,12 \text{ kg/km}$), economia financeira ($\text{km} \times \text{custo_por_km}$)
- Seção "Meu Impacto" adicionada à tela de perfil com três métricas visuais: km compartilhados, CO2 evitado (kg) e economia acumulada (R\$)
- Badge "Eco-Amigo" já existente atualizado para usar os dados do endpoint
- Cálculo baseado no histórico de caronas concluídas — tanto como motorista quanto como passageiro

Para considerar pronto:

Critério de Aceitação	Status
Seção "Meu Impacto" visível no perfil com km, CO2 e economia acumulada	■ Planejado
Cálculos corretos validados com dados do seed de demonstração	■ Planejado
Badge "Eco-Amigo" atualizado para usar os dados do endpoint eco-stats	■ Planejado

US25 — Exportação de comprovante em PDF (2 pontos)

Como usuário, quero exportar um comprovante em PDF de uma carona concluída para usar como comprovante de deslocamento.

Como será feito:

- Endpoint GET /api/caronas/:id/comprovante — gera PDF com dados da carona (origem, destino, data, motorista), lista de passageiros e valor pago
- Biblioteca PDFKit para geração do PDF no backend — sem dependências externas pesadas
- Layout com logo do UniCaronas, dados formatados e rodapé com data de emissão e número de referência único
- Botão "Exportar comprovante" exibido na tela de detalhes para caronas com status "concluída" — visível para motorista e passageiros
- Arquivo gerado dinamicamente e enviado como download — sem armazenamento no servidor

Para considerar pronto:

Critério de Aceitação	Status
Botão "Exportar comprovante" visível em caronas concluídas para os participantes	■ Planejado
PDF gerado com dados corretos — origem, destino, data, motorista e valor	■ Planejado
Arquivo baixado diretamente pelo navegador sem armazenamento no servidor	■ Planejado

6. Tecnologias

Tecnologia	Para que será usada
HTML, CSS e JavaScript	Seção Eco-Stats no perfil, formulário de inserção de código/token e botão de exportação de comprovante
Node.js + Express	Middleware de validação de domínio, geração e verificação de token, camada de serviços refatorada e endpoint eco-stats
Nodemailer + SMTP gratuito	Envio do código/token de verificação ao e-mail institucional do usuário (Gmail, Outlook ou Brevo free tier) — sem custo
Lista local de domínios	JSON (domains.json) ou tabela PostgreSQL dominios_permitidos — armazenamento e consulta de domínios institucionais reconhecidos sem API externa
PDFKit	Geração de comprovantes em PDF no backend com layout do UniCaronas
Jest + Supertest	Testes unitários dos serviços (incluindo validação de domínio e token) e testes de integração do fluxo completo
ESLint	Linting e padronização de código em todo o repositório
PostgreSQL	Campos email_verificado e instituicao_nome em usuarios; tabela verification_tokens para controle de tokens
GitHub	Versionamento — entrega incremental com commits por funcionalidade

7. Quando será testado

Os testes são manuais e integrados ao desenvolvimento diário, com ciclo por semana.

Data	O que é testado	Como é testado
13/05	US22 — Validação por token de e-mail	Cadastrar com e-mail institucional válido → verificar recebimento do token → confirmar → checar email_verificado = true; tentar cadastrar com e-mail pessoal → verificar erro 400
13/05	US23 — Refatoração + testes	Rodar npm test e verificar cobertura >= 70%; executar ESLint e confirmar zero warnings
14/05	US24 — Eco-Stats	Acessar perfil de usuário com histórico de caronas e verificar km, CO2 e economia calculados corretamente
14/05	US25 — Comprovante PDF	Clicar em "Exportar comprovante" em carona concluída e verificar download do PDF com dados corretos e logo do UniCaronas
14/05	Fluxo completo + correções finais	Cadastro com e-mail institucional → envio de token → confirmação → login → criar carona → solicitar → aceitar → pagar → concluir → exportar comprovante → ver Eco-Stats. Product Owner valida cada etapa. Resolver bugs encontrados; atualizar README com SMTP_USER, SMTP_PASS, SMTP_HOST, SMTP_PORT e instruções de teste.

8. Quando vai para produção

Nesta Sprint, 'ir para produção' significa ter o código refatorado, testado e disponível na branch principal do GitHub.

Data	O que acontece
14/05 (Qua)	Código finalizado e integrado na branch principal — versão estável, testes passando e ESLint sem warnings
14/05 (Qua)	Apresentação das funcionalidades do Sprint 7 — demonstração da validação por token de e-mail, Eco-Stats, comprovante PDF e resultados de cobertura de testes

9. Visão Geral do Backlog — 2º Bimestre

O quadro abaixo consolida os itens do Roadmap 2º Bimestre com status atual.

ID	Funcionalidade	Story Points	Prioridade	Sprint	Status
US19	Central de notificações em tempo real	5 SP	Alta	Sprint 6	■ Entregue
US20	E-mail de resumo semanal	3 SP	Média	Sprint 6	■ Entregue

US21	Indicador de digitação no chat	2 SP	Média	Sprint 6	■ Entregue
US22	Validação de e-mail por código/token	5 SP	Alta	Sprint 7	■ Planejado
US23	Refatoração do código + testes	5 SP	Alta	Sprint 7	■ Planejado
US24	Dashboard de Impacto (Eco-Stats)	3 SP	Média	Sprint 7	■ Planejado
US25	Exportação de comprovante em PDF	2 SP	Média	Sprint 7	■ Planejado
US26	Ponto de encontro flexível / busca por raio	5 SP	Média	Backlog	— Backlog
US27	Mural de caronas por curso	3 SP	Baixa	Backlog	— Backlog
US28	Carona de Emergência	3 SP	Média	Backlog	— Backlog
US29	Ranking de Sustentabilidade	2 SP	Baixa	Backlog	— Backlog